Serverless Control Plane, Container Data Plane: A Governance Boundary for Multi-Tenant SaaS Platforms

Shayan Ghasemnezhad, Giacinto Paolo (GP) Saggese, Paul Smith, Heanh Sok, and Vedanshubhai Joshi

***Abstract*—Multi-tenant AI platforms often spread governance through every service. One handler validates identity, another trusts a schema prefix, and a generated API spec exposes modules a tenant never licensed. We present a production pattern that moves identity materialization, tenant scope derivation, module gating, API-surface filtering, and auditable view-as support into a serverless control plane. Container workloads on Kubernetes, Lambda-backed modules, and Fargate jobs execute only inside injected scope. In eight months at Causify AI, the pattern concentrated shared governance in a 135-line Lambda middleware spine, covered 12 data-plane services, moved five of six tenant lifecycle operations out of deployments, and reduced visible API surface by 67–90% for limited-module tenants. Full materialization was 856 ms P50; ETag revalidation handled 91% of steady-state traffic at 68 ms P50.**

***Index Terms*—multi-tenant SaaS, authorization, serverless architecture, control plane, scope injection, confused deputy, API governance, AWS Lambda, Kubernetes**

I. Introduction

Multi-tenant platforms grow by adding tenants, modules, support workflows, and workloads. Each addition expands the governance surface: who is calling, which tenant context is effective, which modules are licensed, which routes should exist, and which database scope a job should use. The usual path is local convenience. A service validates one claim. A worker trusts one parameter. A generated API document exposes one extra module. The product keeps moving, while governance quietly spreads into every corner of the runtime.

This paper describes a production architecture built around a hard boundary.

- **Serverless control plane:** owns governance decisions.
- **Container data plane:** executes workloads inside injected scope.

The boundary is structural, not aspirational. The data plane does not own tenant lookup, module registry, API-surface filtering, or impersonation allow-lists. Those live in the control plane. The AWS SaaS Lens recommends control/data-plane separation for SaaS architectures [1]; this paper shows one concrete way to make the separation load-bearing.

The contributions are practical:

- Three recurring governance failures: scope ambiguity, governance scatter, and surface drift.
- A control-plane design that combines OIDC-style identity materialization, ABAC-style scope derivation, module-gated routing, OpenAPI filtering, and auditable view-as.
- Production evidence across 12 data-plane services, including latency, endpoint exposure, lifecycle-operation cost, and audit limitations.

II. Governance Failures

Scope ambiguity. Services receive tenant identifiers from callers and treat them as truth. OWASP rates broken object-level authorization as the top API security risk [2]; NIST SP 800-162 recommends authorization decisions derived from verified attributes, not caller assertions [3]. The gap between those principles and "trust the header" implementations produces tenant-confusion incidents.

Governance scatter. Every new service copies some version of identity resolution, module gating, and audit logging. With 12 data-plane services, one governance rule change becomes 12 changes. That is how policy turns into folklore.

Surface drift. Generated API documents often expose every route the platform knows, not every route the effective tenant may use. Clients discover disabled modules, call them, and get errors. Security sees attack surface. Support sees tickets.

These failures motivate five design goals:

- **G1 Explicit tenancy:** actor and effective tenant are always distinct fields.
- **G2 Scope injection:** scope flows downstream from the control plane, never upstream from callers.
- **G3 Minimized surface:** tenants see only the endpoints they have licensed.
- **G4 Auditable impersonation:** support can view as a tenant without shared credentials.
- **G5 Governance as data:** routine policy changes do not require code deployments.

III. Architecture

Figure 1 shows the boundary. The control plane is API Gateway [4], Lambda [5], Cognito, and DynamoDB. The data plane is EKS services, module Lambdas, Fargate jobs, and tenant-scoped storage.

The identity contract. The control plane exposes `/v2/me`, a versioned contract similar to an OpenID Connect UserInfo endpoint [6]. It returns actor identity, effective context, derived signals such as `schema_prefix`, and allowable impersonation targets. The schema prefix belongs to the effective context. When support uses view-as, the actor remains the support engineer; the effective tenant changes.

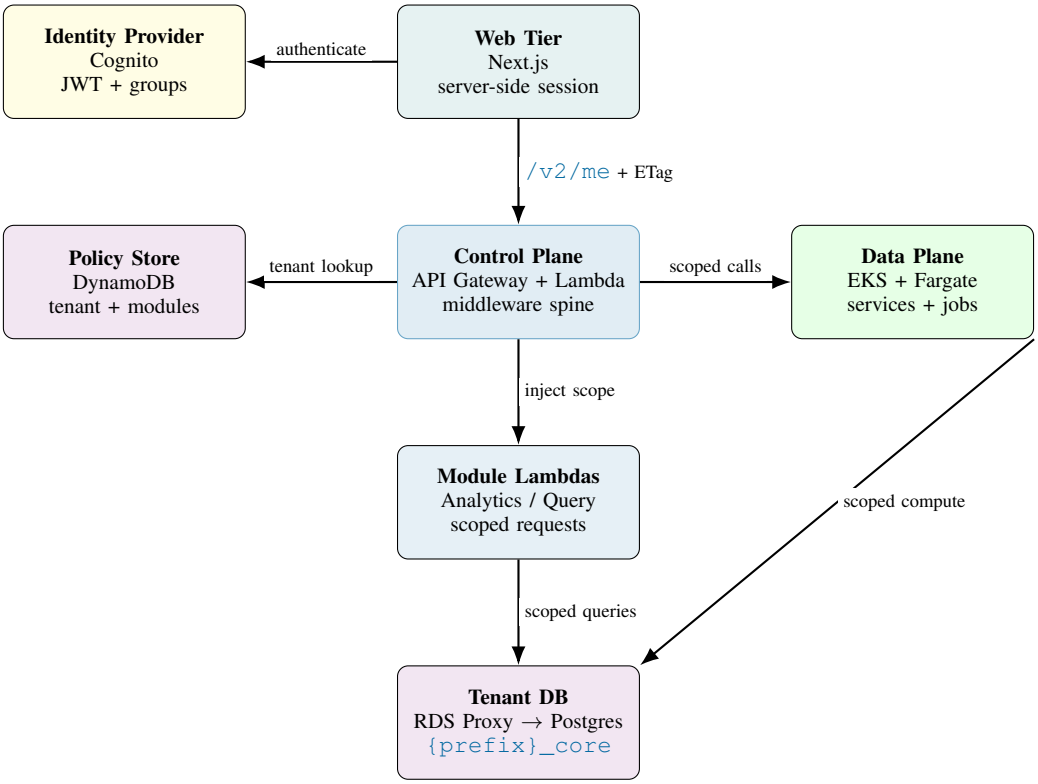

Fig. 1: The control plane resolves identity, effective tenant context, module capability, API surface, and downstream scope. The data plane executes within injected scope.

The endpoint supports ETag and `If-None-Match` semantics under RFC 9110 [7]. Identity is cheap to revalidate and expensive only when it changes.

IV. Control Plane Design

The Lambda is built around four pillars.

Middleware spine. All handlers pass through the same authentication, response-header, and ETag middleware chain. The spine validates Cognito JWTs, resolves tenant membership from the platform table, derives role and `schema_prefix`, builds impersonation allow-lists, and applies security headers.

Scope injection. Tenant scope is derived exclusively from the tenant record, never from headers or query parameters. The control plane injects `schema_prefix` into downstream Lambda and job payloads. Downstream services reject missing or malformed scope; they do not decide which tenant a caller belongs to.

Module gating and surface minimization. Each route carries a module requirement such as `@requires_module("grid")`. The control plane rejects missing modules before handler execution. The `/v1/openapi.json` endpoint filters paths by the effective tenant's enabled-module set and prunes unreferenced schemas.

Auditable impersonation. The view-as mechanism lets operators view as a tenant without shared accounts. The `X-View-As` payload is allow-listed server-side, signed with HMAC-SHA256, bound to a nonce, stored in an encrypted server-side session, and revoked on logout or tenant switch. The browser never receives the signing key. If nonce registration fails, activation fails closed.

V. Data Plane Integration

The data plane runs Amazon EKS services, module Lambdas, Airflow-driven workflows, and ECS/Fargate jobs. Long-running workloads include Optima-style portfolio optimization, DataMap pipeline orchestration, and scheduled analytics. The control plane calls into the data plane; the data plane does not call back to resolve governance.

Database credentials are managed through AWS Secrets Manager and RDS Proxy. Per-tenant isolation uses PostgreSQL schemas. Module Lambdas set `search_path = '{prefix}_core'` after validating `schema_prefix`. Validation protects against malformed strings. It does not replace authoritative derivation.

Fargate jobs receive tenant scope at launch through the ECS `RunTask` payload [8]. No per-job identity resolution occurs. Jobs are data-plane executions with longer time budgets, not exceptions to the governance model.

VI. Defense-in-Depth

The threat model focuses on forged tenancy context, confused-deputy scope substitution [9], disabled-endpoint discovery, and operator damage during impersonation. Five controls make those failures harder.

1) JWT validation is required, but tenant scope is re-derived from membership state.
2) view-as is allow-listed, signed, nonce-bound, and attributable.
3) Scope is injected downstream and missing scope is rejected.

4) Unlicensed endpoints disappear from the served API spec and return `403` if called directly.
5) Network and transport hardening prevent public shortcuts around compute and data stores.

The policy store is now a critical surface. That is the trade. Moving governance from code to data makes tenant operations faster, but a bad policy write can hurt. Policy-store writes therefore need scoped IAM, review, schema validation, and audit logs.

VII. Evaluation

All measurements come from one production deployment at Causify AI.

Latency. Table I reports `/v2/me` full-materialization latency from 1,368 production observations at 512 MB Lambda memory, collected from CloudWatch metrics during March–April 2026.

TABLE I: `/v2/me` full-materialization latency at 512 MB.

Metric	ms
P50	856
P75	938
P90	1,038
P95	1,112
P99	1,176

ETag revalidations return `304 Not Modified` at 68 ms P50 and account for 91% of steady-state requests. At that hit rate, amortized per-page cost is about 139 ms. Cold starts add 300–600 ms in observed tail cases; Provisioned Concurrency is available when needed [5].

Scope confusion. CI checks that route handlers apply module requirements and that Lambdas receiving `schema_prefix` reject missing values. Runtime logs emit actor tenant, effective tenant, and schema prefix. Within this detection envelope, structured-log correlation found zero actor/effective-tenant mismatches over eight months of production operation. The method cannot detect a wrong tenant derivation if the control plane derives the wrong value consistently.

Surface reduction. Table II shows endpoint exposure across synthetic module tiers from a 48-endpoint surface.

TABLE II: Endpoint exposure after OpenAPI filtering.

Modules enabled	Visible	Reduction
0 (core only)	5	90%
1	16	67%
2	27	44%
3+	38	21%
Full access	48	0%

Governance concentration. Shared governance lives in a 135-line Lambda middleware spine. Reimplementing comparable logic across 12 services would place roughly 1,620 lines on the wrong side of the boundary. Five of six common tenant lifecycle operations are policy-store updates taking under five minutes. Adding a new module type still requires code. Migration of the 12 services took roughly 160 engineer-hours across two sprints.

VIII. Related Work and Limits

NIST ABAC [3], OPA [10], Cedar [11], and Zanzibar [12] provide authorization models and policy systems. Our design is narrower. It materializes tenant context once and injects scope into services that should not rediscover it.

Service meshes such as Istio and Linkerd enforce network-layer authorization [13]. They are valuable but do not replace tenant-aware OpenAPI filtering, module-gated routing, or database schema-prefix injection.

The limits are real. The implementation is AWS-specific. The decorator approach is Python-specific. Surface filtering is REST-first. The evaluation is a single-organization case study. And the control plane remains load-bearing: if it derives scope incorrectly, downstream services cannot prove the mistake.

IX. Conclusion

The bug that hurts a multi-tenant platform is often ordinary: a missing scope field, a stale spec, an endpoint that exists when it should not, a job that runs under the wrong tenant. The platform does not collapse. It just becomes less trustworthy.

A serverless control plane paired with a container data plane gives those failures one place to be prevented, reviewed, and logged. The control plane decides and attests. The data plane executes. That boundary is not a silver bullet, but it made governance less scattered, tenant changes cheaper, API exposure smaller, and incident analysis calmer.

Takeaways for Practitioners

- **Materialize identity as a versioned contract.** Expose actor, effective tenant, modules, impersonation, and scope in one response. Cache with ETag/304.
- **Derive and inject scope.** Resolve `schema_prefix` from tenant state and reject downstream calls that omit it.
- **Filter API surface at the boundary.** Prune OpenAPI paths by enabled modules. Keep backend authorization as the final barrier.

References

[1] Amazon Web Services, "SaaS Lens - AWS Well-Architected Framework," Documentation, accessed 2026-07-15. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/welcome.html

[2] OWASP Foundation, "OWASP Top 10 API Security Risks - 2023," Website, accessed 2026-07-15. [Online]. Available: https://owasp.org/API-Security/editions/2023/en/0x11-t10/

[3] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "NIST SP 800-162: Guide to Attribute Based Access Control (ABAC) Definition and Considerations," National Institute of Standards and Technology, 2014. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-162

[4] Amazon API Gateway, "Amazon api gateway documentation," Documentation, accessed 2026-07-15. [Online]. Available: https://docs.aws.amazon.com/apigateway/

[5] AWS Lambda, "Aws lambda documentation," Documentation, accessed 2026-07-15. [Online]. Available: https://docs.aws.amazon.com/lambda/

[6] OpenID Foundation, "OpenID Connect Core 1.0 incorporating errata set 2," Specification, accessed 2026-07-15. [Online]. Available: https://openid.net/specs/openid-connect-core-1_0.html

[7] R. T. Fielding, M. Nottingham, and J. Reschke, "RFC 9110: HTTP semantics," RFC Editor, 2022, accessed 2026-07-15. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110.html

[8] Amazon Elastic Container Service, "RunTask - Amazon Elastic Container Service API Reference," Documentation, accessed 2026-07-15. [Online]. Available: https://docs.aws.amazon.com/AmazonECS/latest/APIReference/API_RunTask.html

[9] N. Hardy, "The confused deputy (or why capabilities might have been invented)," in *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, 1988, pp. 36–38.
[10] Open Policy Agent Contributors, "Open policy agent documentation," Website, accessed 2026-07-15. [Online]. Available: https://www.openpolicyagent.org/docs/
[11] Cedar Policy Language Authors, "Cedar policy language reference guide," Website, accessed 2026-07-15. [Online]. Available: https://docs.cedarpolicy.com/
[12] R. Pang, M. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner *et al.*, "Zanzibar: Google's consistent, global authorization system," in *Proceedings of the USENIX Annual Technical Conference*, 2019. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/pang
[13] Istio Authors, "Authorization policy," Documentation, accessed 2026-07-15. [Online]. Available: https://istio.io/latest/docs/reference/config/security/authorization-policy/

About the Authors

Shayan Ghasemnezhad is Infrastructure and DevOps Lead at Causify AI, Turin, Italy. His research interests include platform engineering and cloud solutions architecture, production AI/ML systems, and secure, reliable infrastructure operations. Ghasemnezhad received his Bachelor's degree in Digital Management from Ca' Foscari University of Venice. Contact him at shayan@causify.ai.

Giacinto Paolo (GP) Saggese is Co-founder and CTO at Causify AI, Potomac, Maryland, USA. His research interests include machine learning, fintech, and the commercialization of research. Saggese received his Ph.D. degree in Electrical and Computer Engineering from the University of Illinois Urbana-Champaign. He is an Adjunct Professor at the University of Maryland, College Park. Contact him at gp@causify.ai.

Paul Smith is Co-founder and Chief Scientist at Causify AI, Cary, North Carolina, USA. His research interests include causal inference, Bayesian modeling, and time-series prediction. Smith received his Ph.D. degree in Mathematics from UCLA. Contact him at paul@causify.ai.

Heanh Sok is Software Engineer at Causify AI, College Park, Maryland, USA. His research interests include big data systems, distributed systems, and AI/ML engineering and operations. Sok received his Master's degree in Data Science from the University of Maryland, College Park. Contact him at h.sok@causify.ai.

Vedanshubhai Joshi is Software Engineer at Causify AI, Indiana, USA. His research interests include platform infrastructure, multi-tenant API design, and data engineering pipelines. Joshi received his Master's degree in Computer Science from Purdue University. Contact him at v.joshi@causify.ai.